

AA5. Casos de uso

Autor: Pol Valle

Año: 2026

Índice

Contenido

Introducción	3
Evolución del Programa	3
Diagrama de casos de uso	3
AA3 - Gestión de inventario	3
AA4 - Gestión de clientes y ventas	4
AA5 - Resumen agregado de ventas	5
Decisiones de diseño	6
Implementación en Java	7
Comentarios sobre el desarrollo	7
Mayor dificultad	7
No completado	7
Mejoras posibles	8
Video	8
Autoevaluación	8

Introducción

Esta actividad va de casos de uso en WoodShops.

Sobre lo que se hizo en AA4, se ha añadido la funcionalidad de mostrar un resumen de ventas por tienda entre fecha de inicio y de finalización.

Además se han elaborado los diagramas de casos de uso con actores, relaciones, precondiciones y post condiciones para las funcionalidades de las 3 actividades.

Evolución del Programa

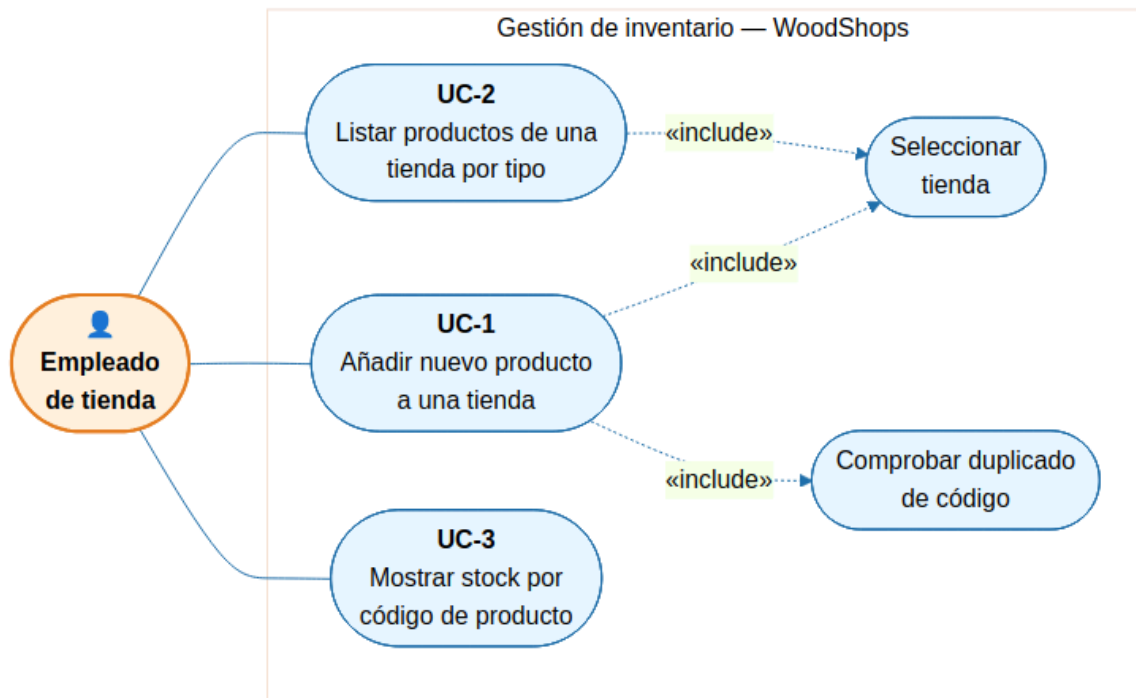
Respecto a la actividad anterior, los cambios son:

- Nueva opción (opción 8): Resumen de ventas por tienda (entre fechas).
- Nuevo método Tienda.getTotalEntre(inicio, fin) que encapsula el cálculo del total de ventas de una tienda en un rango.
- Nuevo método resumenVentasPorTienda en MainWoodsShops, que pide el rango de fechas, las valida y recorre las tiendas pidiendo a cada una su total.

Diagrama de casos de uso

AA3 - Gestión de inventario

El único actor es el **empleado de tienda**, que tiene tres operaciones disponibles. En **UC-1 (Añadir producto)** elige una tienda y el sistema valida que el código no esté duplicado antes de guardar. En **UC-2 (Listar productos por tipo)** elige tienda y filtra por tablero, barniz o artículo. En **UC-3 (Mostrar stock por código)** introduce un código y el sistema lo busca en todas las tiendas, sin necesidad de seleccionar ninguna — por eso este caso no tiene includes.

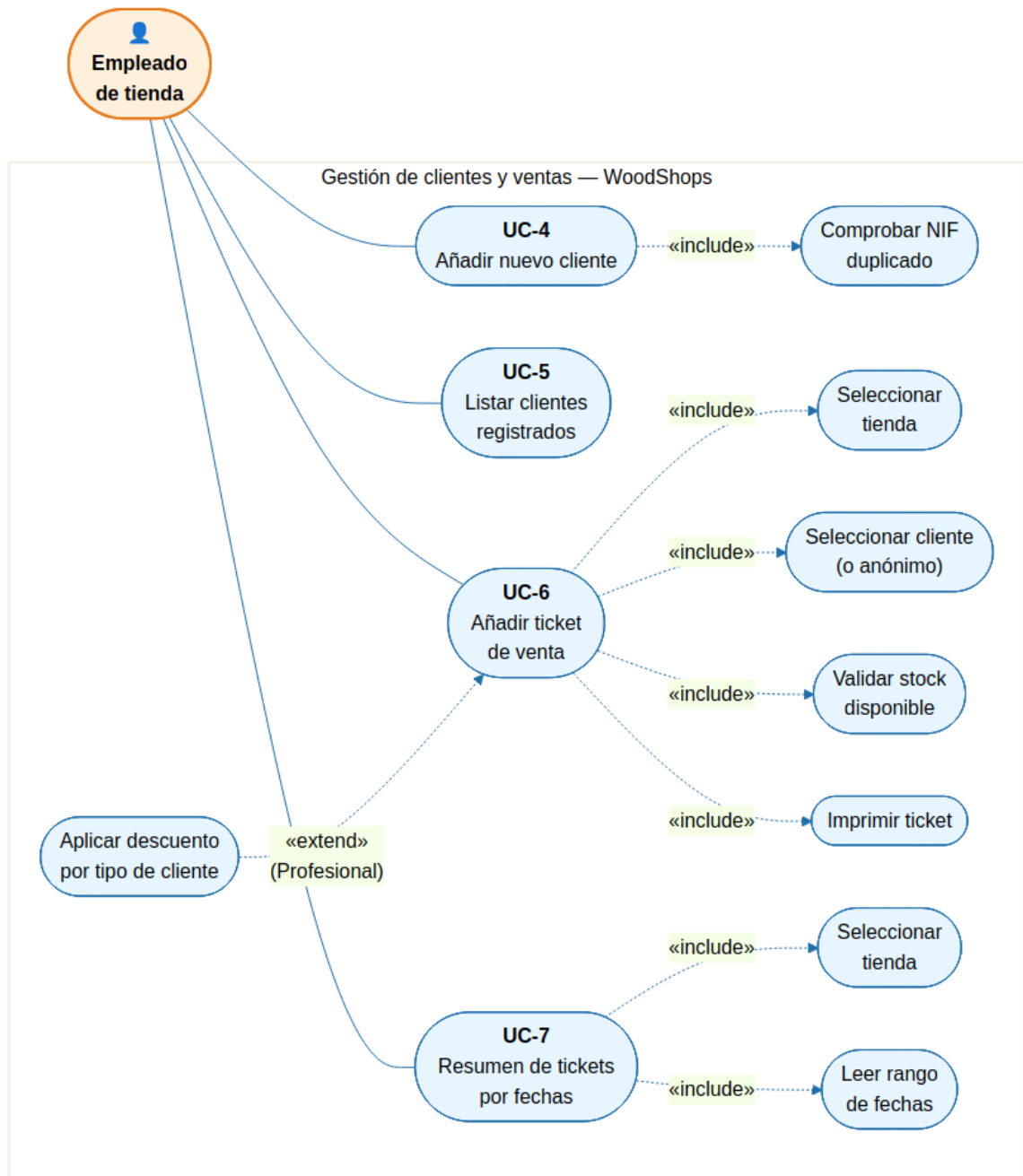


AA4 - Gestión de clientes y ventas

El actor principal es el **empleado de tienda**, que es quien maneja la consola; el cliente final nunca interactúa con el sistema, así que no figura como actor.

UC-4 (Añadir cliente) comprueba que el NIF no exista antes de dar de alta. **UC-5 (Listar clientes)** simplemente muestra la lista. **UC-6 (Añadir ticket)** es el más completo: elige tienda y cliente, valida stock y al final imprime el ticket. **UC-7 (Resumen de tickets)** muestra los tickets de una tienda entre dos fechas.

El descuento aparece como «extend» porque solo entra en juego con clientes Profesionales — los WoodFriend y los anónimos no disparan ese flujo.

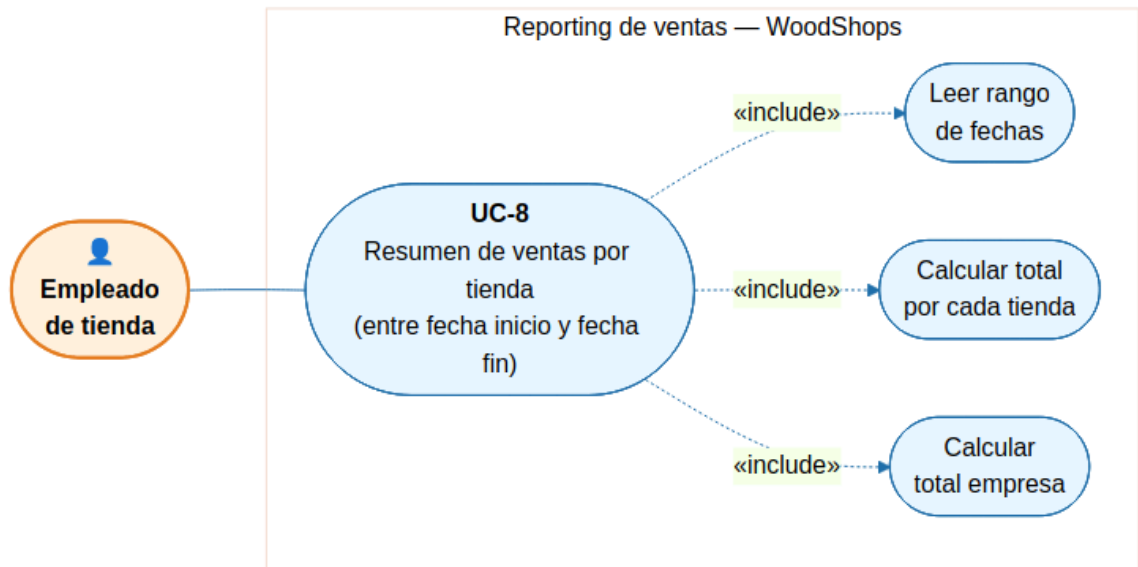


AA5 - Resumen agregado de ventas

El actor sigue siendo el **empleado de tienda**, ahora con una vista global. En **UC-8 (Resumen de ventas)** introduce un rango de fechas y el sistema recorre todas las tiendas mostrando lo facturado por cada una más el total de la empresa.

Los tres pasos —leer fechas, calcular total por tienda y calcular total empresa— son «include» y no «extend» porque los tres ocurren siempre, sin excepción.

Precondiciones: al menos una tienda registrada y $\text{fin} \geq \text{inicio}$. **Postcondición:** se muestra el desglose por tienda y el total agregado.



Decisiones de diseño

getTotalEntre en Tienda (no en Main). La lógica de agregación vive ahora en la clase de dominio. El Main solo pregunta y muestra. Sigue el patrón Tell, Don't Ask y abre la puerta a reutilizar el cálculo desde otras vistas (por ejemplo, una hipotética opción "top-N tiendas").

Reutilización de Ticket.getTotal(). El total ya incluye el descuento por cliente Profesional, por lo que el resumen de ventas no duplica esa lógica. Si en el futuro se modifica la regla de descuento, basta tocar Ticket.

Precio congelado en LineaTicket. Heredado de AA4. Si después de una venta cambia el precio del producto en el inventario, los totales históricos no se ven afectados. Esto hace el resumen de ventas determinista.

Validación de fechas en una sola función (leerFecha). Reutilizada tanto en el resumen de tickets (AA4, opción 7) como en el resumen de ventas por tienda (AA5, opción 8). Devuelve null si el formato no es válido, lo que simplifica el flujo de cancelación.

Descuento como «extend», no «include». El descuento solo se aplica si el cliente es Profesional. Modelarlo como inclusión obligaría a un nodo "calcular descuento" que la mayoría de tickets no usaría. La extensión refleja el comportamiento real.

Implementación en Java

Prácticas aplicadas:

- Validación de duplicados (existeCodigoEnTienda, existeNifCliente).
- Validación de rangos en descuentos e índices de enums.
- Captura de excepciones (InputMismatchException, DateTimeParseException).
- Menús genéricos sobre enums con leerEnum<E extends Enum<E>>.
- Pausa y limpieza de consola entre opciones (pausarYLimpiar) para mejorar legibilidad.
- Javadoc completo en todas las clases y métodos públicos.

Comentarios sobre el desarrollo

Mayor dificultad

Refactorizar el cálculo agregado: tras tener funcionando la opción 8 con la lógica en el Main, moverla a Tienda.getTotalEntre() exigió revisar que el comportamiento (rangos inclusivos, descuentos aplicados) se mantuviera idéntico.

Decidir entre «include» y «extend» en los diagramas. Modelar el descuento como «extend» (no siempre se ejecuta) en lugar de «include» fue la decisión que más he discutido conmigo mismo.

No completado

Ningún punto funcional del enunciado se ha dejado fuera.

Mejoras posibles

Clase utilitaria Consola Utils para centralizar los try/catch

Tests unitarios con JUnit 5 sobre `Tienda.getTotalEntre()`, `Ticket.getTotal()` y `LineaTicket.getSubtotal()`

Video

https://youtu.be/hhPVr_qVkml

Autoevaluación

Criterio	Justificación	Puntos
Realización de diagramas de uso	Tres diagramas separados (AA3, AA4, AA5) con actores principales y secundarios, «include» y «extend» aplicados con criterio, y notas con precondiciones y postcondiciones.	7/10
Generación de código	Herencia. Polimorfismo, enums, validaciones	7/10

Nota global autoasignada: 70/100.